UNIVERSITY OF CALIFORNIA

SANTA CRUZ

SATURN: ZERO-CONFIGURATION AI SERVICE DISCOVERY

A thesis submitted in partial satisfaction

of the requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Joseph Perrello

March 2026

The Thesis of Joseph Perrello

is approved:

Adam Smith, Chair

Ram Sundara Raman

Peter Biehl

Vice Provost and Dean of Graduate Studies

Copyright © by

Joseph Perrello

2026

Contents

List of Figures **ix**

List of Tables **x**

Abstract **xi**

Acknowledgments **xii**

1 Introduction **1**

1.1 The Vision: AI as Network Infrastructure 2

1.2 Thesis Statement 3

1.3 Saturn and This Thesis 4

2 Background **6**

2.1 Service Discovery on Local Networks 6

2.1.1 NetBIOS 7

2.1.2 Universal Plug and Play 7

2.1.3 DLNA 7

2.1.4 WS-Discovery 8

2.1.5 Why mDNS/DNS-SD 8

2.1.6 DHCP 9

2.2 mDNS and DNS-SD 10

2.2.1 Multicast DNS 10
2.2.2 DNS-Based Service Discovery 10
2.2.3 TXT Records and Metadata 11
2.2.4 Implementations: Bonjour and Avahi 11
2.2.5 Adoption in New Domains 12
2.2.6 Privacy Considerations 13
2.3 The AI Service Landscape 13
2.3.1 Hosted Chat Applications 13
2.3.2 Coding Agents and Assistants 14
2.3.3 Voice Typing and Transcription 14

3 Design **16**
3.1 Goals 16
3.2 Audiences 17
3.3 Concepts 18
3.3.1 Endpoints 18
3.3.2 Beacons 19
3.3.3 Priorities 19
3.3.4 Ephemeral Keys 20
3.4 Protocol Specification 21
3.4.1 Service Type 21
3.4.2 Record Structure 21
3.4.3 Endpoint Expectations 23
3.4.4 Discovery Process 23
3.5 Architecture Decisions 24

4 Implementation **26**

4.0.1 Server Architecture 26
4.0.2 Client Discovery 26
4.1 Server Types 27
4.1.1 Ollama Server 27
4.1.2 OpenRouter Server 27
4.1.3 Fallback Server 27
4.2 Client Integration Patterns 28
4.2.1 Native SDK Integration 28
4.2.2 Application Plugin 28
4.2.3 Tool Protocol Bridge 28
4.2.4 Subprocess Bridge 28
4.3 GL.iNet Router Deployment 31
4.3.1 Hardware Constraints 31
4.3.2 OpenWRT Integration 31
4.3.3 Significance 31
4.3.4 Limitations 32

5 Evaluation **33**

6 Discussion **34**

7 Conclusion **35**

A Saturn Protocol Specification **36**
A.1 Service Registration 36
A.2 TXT Record Schema 37
A.3 Endpoint Requirements 37
A.4 Discovery Algorithm 37

A.5 Ephemeral Key Rotation Protocol 39

A.6 Conformance 40

B Generative AI in Thesis Development 42

B.1 Knowledge Graph Authorship 42

B.2 Iterative Advisory Role 43

B.3 Limits and Human Judgment 45

List of Figures

List of Tables

3.1 Saturn TXT Record Schema 22

3.2 Required Saturn Endpoint Paths 23

A.1 Complete TXT Record Field Reference 41

Abstract

Saturn: Zero-Configuration AI Service Discovery

Joseph Perrello

Generative AI services require users to obtain API keys, manage credentials, and configure endpoint URLs for each application and provider. These per-user, per-application configuration steps create barriers to access that disproportionately affect resource-constrained populations. This thesis presents Saturn, a protocol for zero-configuration AI service discovery on local networks using multicast DNS (mDNS) and DNS-based Service Discovery (DNS-SD).

Saturn servers advertise OpenAI-compatible AI endpoints with ephemeral, time-limited credentials on the local network. Clients discover these services automatically without manual configuration. We advance three claims: (1) zero-configuration discovery is feasible via mDNS/DNS-SD, (2) network-level provisioning reduces configuration effort compared to manual setup, and (3) information exposure is documented and bounded.

We demonstrate Saturn across six implementations spanning four programming languages and show practical deployment on consumer hardware. The protocol extends zero-configuration networking—proven for printers and speakers—to generative AI services, enabling institutions to provision shared AI access at the network level and reducing total configuration burden from scaling with the number of users to scaling only with the number of administrators.

Acknowledgments

I thank Adam Smith for patient advising and clear feedback throughout this work. Ram Sundara Raman provided critical perspective on network security. Peter Biehl and the graduate division enabled this research through institutional support.

The mDNS protocol exists because of decades of work by the IETF DNSSD working group, particularly Stuart Cheshire and Marc Krochmal. Without their prior art, Saturn would have required inventing discovery rather than assembling existing infrastructure.

This work emerged from frustration: seeing brilliant students and researchers blocked from accessing AI tools they needed, not by capability but by credential management and configuration friction. Saturn exists because that situation is fixable. I hope it helps.

Chapter 1

Introduction

In 1954, Lewis Strauss predicted that atomic energy would become "too cheap to meter" [37]. Seventy years later, generative AI carries a similar promise—and a similar gap between rhetoric and reality. ChatGPT Plus costs $20 per month [31]. Anthropic's Claude Pro costs the same [1]. Users who need multiple providers or higher usage limits pay $100 or more per month. These are not edge cases; they represent the standard path to accessing the most capable AI systems available today.

The cost barrier alone would warrant attention, but access to generative AI demands more than a subscription. Each application that consumes AI must be configured independently: an API key obtained from a provider, an endpoint URL, and often a specific model identifier. Syed et al. [39] document this as the standard architecture of AI-as-a-Service, where platforms expose REST APIs behind authentication tokens that individual users must obtain, manage, and rotate. A developer using three AI-powered tools—a chat interface, a coding assistant, and a transcription service—must configure credentials for each one separately.

This per-user key management model is not merely inconvenient; it is broken. Meli et al. [24] found that over 100,000 public GitHub repositories contained leaked API

secrets, with 81% of those secrets never revoked even after notification. The best automated scanning tools they evaluated detected only 25% of leaked credentials. Static API keys, the dominant access mechanism for AI services, create a class of security vulnerabilities that current tooling cannot adequately address.

The consequences fall unevenly. Bassignana et al. [4] demonstrate that socioeconomic status correlates with the quality of interactions people have with language technology—those with fewer resources extract less value even when they gain access. Gabriel [15] identifies generative AI as a widening force in educational inequality, where students who can afford subscriptions gain compounding advantages in writing, research, and problem-solving. Capraro et al. [6] find that generative AI's economic benefits accrue disproportionately to those who already hold advantages. Costa et al. [12] frame accessibility and usability as foundational dimensions of AI democratization, arguing that technical barriers reinforce existing power asymmetries. The current model—individual subscriptions, per-app configuration, manual credential management—excludes precisely the populations most likely to benefit from shared access.

1.1 The Vision: AI as Network Infrastructure

When Apple introduced Bonjour in 2002, a user who connected to a WiFi network could print within seconds—no driver downloads, no IP addresses, no configuration dialogs. The underlying protocol, Multicast DNS (mDNS) combined with DNS-based Service Discovery (DNS-SD), allowed printers to announce their presence on the local network and clients to find them automatically. Guttman [17] described the goal of zero-configuration networking as enabling "direct communications between two or more computing devices via IP" with no manual setup, and RFC 1122 [5] articulated

the principle even earlier: “It would be ideal if a host implementation of the Internet protocol suite could be entirely self-configuring.”

Generative AI services have no equivalent. Every AI application today requires users to manually locate a provider, create an account, obtain credentials, and configure each tool individually. Yet the technical requirements for discovery are similar to those that printers solved twenty years ago: a service exists on a network, it has capabilities that clients need to know about, and the connection requires minimal metadata.

Institutional networks already provision shared resources this way. University eduroam networks grant students WiFi access across hundreds of campuses without per-institution accounts. Campus print services discover printers automatically. The conceptual step from “join the network, get the printer” to “join the network, get the AI” is small. The implementation gap is what Saturn addresses.

1.2 Thesis Statement

We can and should provision generative AI at the network level. Saturn demonstrates this is technically achievable using existing zero-configuration protocols, enabling shared access models where joining a network means gaining AI capabilities. This thesis advances three claims. First, zero-configuration network protocols (mDNS and DNS-SD) can provision AI services without end-user configuration. Second, network-level provisioning reduces total configuration effort by centralizing setup in a single administrator rather than distributing it across every user. Third, the security and privacy trade-offs of broadcast-based AI discovery are documented in existing literature and can be mitigated without destroying the zero-configuration property.

1.3 Saturn and This Thesis

Saturn is a protocol and family of implementations for zero-configuration AI service discovery. Servers advertise OpenAI-compatible AI endpoints on the local network using mDNS/DNS-SD under the service type `_saturn._tcp.local.`; clients discover these services automatically and connect without configuration. The protocol specifies a beacon architecture: servers broadcast metadata about where AI services live, rather than proxying traffic through a central point. This eliminates single points of failure and preserves the direct relationship between client and AI provider.

We implemented Saturn six times across four programming languages. The core Python package provides both server and client libraries. A TypeScript provider integrates with Vercel's AI SDK framework. An Open WebUI plugin adds discovery to an existing chat interface. A Model Context Protocol server exposes Saturn to AI coding assistants. A Lua-to-Python bridge brings discovery into VLC media player. A Rust implementation runs on a GL.iNet travel router alongside the device's own DHCP and DNS services—the same class of hardware that provisions IP addresses now also provisions AI.

The chapters that follow establish the service discovery landscape and AI access patterns that motivate Saturn's design, then present the protocol specification, its six implementations, and deployment scenarios including a design fiction and a 42-application integration analysis. Saturn demonstrates that the infrastructure for shared AI access does not need to be invented—it needs to be assembled from protocols that have worked for decades.

This thesis was developed with substantial support from Claude (Anthropic), a large language model serving two roles: authoring the moons knowledge graph through iterative prompting and providing iterative feedback on coherence and writing. We

document this openly because Saturn argues generative AI should be accessible to everyone—and that commitment demands transparency about how we conducted our research. Concealing AI's role would contradict the equity argument we advance. A full account appears in Appendix B.

Chapter 2

Background

Devices on a local network need to find each other. A laptop locates a printer; a phone finds a speaker; a coding tool connects to a debugger. This problem—service discovery—has generated competing protocols over three decades, each reflecting different assumptions about platform support, scope, and configuration overhead. This chapter examines those protocols, explains why mDNS and DNS-SD fit Saturn's requirements, describes the network layer where Saturn operates, and surveys the AI service landscape Saturn targets.

2.1 Service Discovery on Local Networks

When two devices share a network segment, neither knows the other exists. Service discovery protocols solve this bootstrapping problem by defining how devices announce their presence and how clients search for available services.

2.1.1 NetBIOS

Network Basic Input/Output System (NetBIOS) provided name resolution on local networks before DNS became ubiquitous [16]. Windows machines used NetBIOS over TCP/IP to discover file shares and printers via broadcast messages. NetBIOS remains present in modern Windows networks but has no native support on macOS or Linux. Microsoft began replacing it with mDNS in Windows 10, signaling its trajectory toward deprecation. For Saturn, NetBIOS fails on two counts: it cannot operate cross-platform, and it provides name resolution only—no mechanism for service metadata such as model lists, API types, or feature capabilities.

2.1.2 Universal Plug and Play

Universal Plug and Play (UPnP) combines the Simple Service Discovery Protocol (SSDP) for multicast device discovery with a SOAP/XML framework for device control [29]. Routers use UPnP for automatic port forwarding; media servers use it to stream content. UPnP extends far beyond discovery into device interaction, introducing complexity Saturn does not need, and carries a documented history of security vulnerabilities including remote code execution and unauthorized port forwarding. Apple, Google, and Microsoft have shifted to mDNS/DNS-SD for new service discovery features, reflecting UPnP's declining role.

2.1.3 DLNA

The Digital Living Network Alliance (DLNA) built a media streaming standard on top of UPnP, enabling televisions to discover and play content from network media servers [13]. DLNA inherits UPnP's complexity and restricts its scope to audio and

video. The DLNA consortium dissolved in 2017. Saturn requires general-purpose service metadata exchange, not media transport.

2.1.4 WS-Discovery

Web Services Dynamic Discovery (WS-Discovery) uses multicast SOAP/XML messages for service advertisement, primarily in enterprise environments [26]. Windows uses WS-Discovery alongside mDNS for printer discovery. The protocol's SOAP/XML encoding imposes substantial overhead for lightweight metadata exchange, and its library ecosystem is smaller and less portable than mDNS implementations. WS-Discovery targets managed networks with IT infrastructure—not the consumer and educational environments Saturn serves.

2.1.5 Why mDNS/DNS-SD

mDNS and DNS-SD satisfy Saturn's requirements where the alternatives do not. mDNS resolves names across platforms without infrastructure—no server, no accounts, no configuration. DNS-SD advertises services using standard DNS record types, carrying the metadata Saturn needs in a format every major operating system already supports. The same protocol pair powers AirPrint, AirPlay, Chromecast, and Spotify Connect. Guttman [17] defined zero-configuration networking as enabling "direct communications between two or more computing devices via IP" without manual setup; mDNS/DNS-SD is the dominant realization of that vision. The next section (2.2) explains both protocols in detail.

2.1.6 DHCP

The Dynamic Host Configuration Protocol (DHCP) is Saturn's closest analogy at the network layer [14]. A device joining a network broadcasts a DISCOVER message; a DHCP server responds with an OFFER containing an IP address, gateway, DNS server, and lease duration; the client acknowledges, and connectivity begins without manual configuration.

Saturn and DHCP share a philosophy—join the network, receive the service—but operate at different layers. DHCP runs once, at the moment a device joins the network. It assigns a single IP address along with a gateway, DNS server, and lease duration, then its work is done. The protocol operates at the system level, binding network interfaces with root or administrative privileges, and carries only enough metadata to bring a device online. Saturn's requirements differ in timing, multiplicity, privilege, and payload. AI services appear and disappear dynamically, so discovery must be continuous rather than one-shot. A network may host dozens of services with different models and capabilities, so discovery must enumerate many results rather than a single assignment. Saturn runs in userspace with no elevated privileges and carries rich metadata—model lists, API types, priorities, ephemeral credentials—through DNS records rather than DHCP options.

The two are complementary rather than competing. DHCP configures the network. Saturn configures AI services on that network. On Saturn's GL.iNet router deployment, both run on the same hardware—the device assigns IP addresses via DHCP while simultaneously advertising AI endpoints via Saturn.

2.2 mDNS and DNS-SD

2.2.1 Multicast DNS

Multicast DNS (mDNS), specified in RFC 6762 [10], resolves names on local networks without a DNS server. Rather than querying a designated server, mDNS clients multicast queries to all devices on the link. Any device that owns the queried name responds directly.

Queries travel to the multicast address 224.0.0.251 on port 5353 (ff02::fb for IPv6). Names use the `.local` top-level domain—`my-server.local`, for instance. When a client resolves `my-server.local`, it multicasts the query, and the owning device replies with its IP address. No central authority mediates.

mDNS inherits the DNS record type system: A records (IPv4 addresses), AAAA records (IPv6), SRV records (service locations), TXT records (key-value metadata), and PTR records (service enumeration). The wire format is identical to conventional DNS; only the transport differs with multicast instead of unicast, and link-local scope instead of global.

Siddiqui et al. [35] identify mDNS as one of three pillars of zero-configuration networking, alongside link-local addressing [8] and DNS-SD. Together, these protocols allow devices to obtain addresses, resolve names, and discover services without pre-existing infrastructure.

2.2.2 DNS-Based Service Discovery

DNS-based Service Discovery (DNS-SD), specified in RFC 6763 [9], defines how clients enumerate services by type. While mDNS resolves names to addresses, DNS-SD finds all instances of a given service on the network.

DNS-SD chains three record types. A PTR record maps a service type—such as `_saturn._tcp.local.`—to the names of all instances offering that service. An SRV record for each instance provides hostname and port. A TXT record carries key-value metadata about capabilities. Discovery follows the sequence: query PTR for the service type, receive instance names, then query SRV and TXT for each instance.

DNS-SD can run over mDNS for link-local discovery or over conventional unicast DNS for wide-area use. Saturn uses mDNS transport exclusively, confining discovery to the local network segment.

2.2.3 TXT Records and Metadata

TXT records carry key-value pairs as DNS record data. Each string has a maximum length of 255 bytes, imposed by the DNS wire format. A single TXT record can contain multiple strings, but each must fit within this limit independently.

Saturn uses TXT records to advertise: protocol version, deployment name, API type, base URL, priority level, ephemeral key, rotation interval, and supported features. Clients select among discovered services using this metadata alone, without additional network requests beyond the DNS-SD queries. The 255-byte limit constrains individual field sizes—ephemeral keys must fit within approximately 240 usable characters—but accommodates all metadata Saturn requires.

2.2.4 Implementations: Bonjour and Avahi

Apple released Bonjour in 2002 as the reference mDNS/DNS-SD implementation, bundled with macOS and available for Windows [3]. Bonjour powers AirPrint, AirPlay, and AirDrop. On Linux, the Avahi daemon provides equivalent functionality and ships as a default service on most distributions [33]. Microsoft added native mDNS responder

support in Windows 10.

Every major desktop and mobile operating system can therefore participate in mDNS/DNS-SD discovery without additional software. This cross-platform foundation means any application using system mDNS APIs—regardless of programming language or framework—can discover Saturn services.

2.2.5 Adoption in New Domains

mDNS/DNS-SD originated with printer and file share discovery but has expanded into domains its designers did not anticipate. AirPlay streams audio and video to speakers and televisions. Chromecast casts media from phones and laptops. Spotify Connect discovers playback devices across the local network.

Siljanovski et al. [36] demonstrated that mDNS serves Internet of Things environments, proposing it as a discovery layer for heterogeneous sensor networks. VS Code uses mDNS to discover remote debugging targets, enabling developers to attach debuggers to other machines without entering IP addresses manually [25]. A community contributor proposed adding mDNS service advertisement to Ollama, a widely deployed local AI inference server, in an unmerged pull request [28]—one of the earliest attempts to apply zero-configuration discovery to AI services, and a direct precedent for Saturn's approach.

Saturn continues this trajectory. Where AirPrint discovers printers and AirPlay discovers speakers, Saturn discovers AI endpoints. The service type changes—`_saturn._tcp` instead of `_ipp._tcp`—but the mechanism is identical.

2.2.6 Privacy Considerations

mDNS operates by multicast: every query and response reaches all devices on the link. Kaiser and Waldvogel [20] demonstrated that passive observers can enumerate all services on a network, track device presence over time, and correlate service metadata with users. Könings et al. [22] measured device names in the wild and found that 59% contained real user names, making mDNS announcements a source of personally identifiable information.

These findings apply directly to Saturn. A passive observer on a Saturn-enabled network sees which AI services exist, what models they offer, and potentially who advertises them. Kaiser and Waldvogel [21] proposed privacy-preserving extensions to DNS-SD using hash-based service instance names, which reduce what observers learn without breaking discovery. Chapter 7 analyzes Saturn's specific threat model in the context of these known vulnerabilities.

2.3 The AI Service Landscape

Saturn targets applications that consume AI text completion and transcription services. These applications share a configuration pattern: each requires users to specify an API endpoint, authenticate with credentials, and select a model.

2.3.1 Hosted Chat Applications

Self-hosted chat interfaces provide a chat like experience backed by user-controlled AI endpoints. Open WebUI [30] connects to Ollama [27] instances or any OpenAI-compatible API through a web interface. LibreChat [23] supports multiple providers simultaneously, letting users switch between OpenAI, Anthropic, and local models

within a single session. Jan [18] runs models locally on the desktop while also supporting remote API connections.

Each application requires the user to configure at least one backend: an API base URL, an authentication key, and a model identifier. Users who run multiple chat interfaces multiply this effort. The configuration is static: if a backend changes its address, port, or credentials, every connected application must be updated manually.

2.3.2 Coding Agents and Assistants

AI-powered development tools increasingly support custom endpoints rather than locking users to a single provider. Cursor [2] routes completions through any OpenAI-compatible endpoint, enabling developers to use local models or alternative providers. Continue [11], an open-source coding assistant, connects to arbitrary OpenAI-compatible APIs. Both require users to configure endpoint URLs and API keys per provider.

These tools illustrate the same pattern at a different layer: developers must configure AI access individually for each tool in their workflow, even when all tools ultimately query the same backend.

2.3.3 Voice Typing and Transcription

Voice-to-text tools demonstrate a distinct AI consumption pattern where two models collaborate in a pipeline. Most use an on-device speech recognition model like OpenAI's Whisper [34] to transcribe audio locally. A language model then cleans the raw transcript: correcting grammar, removing filler words, and formatting text.

SuperWhisper [38] runs Whisper entirely on-device, ensuring no audio leaves the user's machine at the cost of accepting whatever quality the local model achieves. Wispr Flow [41] adds a cloud-based language model to clean and reformat transcripts,

delivering polished output for a monthly subscription. MacWhisper [19] offers local transcription with a one-time purchase, avoiding recurring costs but limiting users to bundled models.

This pipeline captures the fragmentation Saturn addresses. A user wanting voice transcription with AI cleanup chooses between a subscription for cloud processing, a standalone tool with fixed capabilities, or manually configuring both a local Whisper instance and a separate language model. Each option encodes infrastructure decisions—which model, where it runs, how to authenticate—into the individual user's configuration.

Chapter 3

Design

Chapter 2 established that mDNS and DNS-SD provide cross-platform, infrastructure-free service discovery, and that every AI application today requires manual endpoint configuration. This chapter specifies how Saturn bridges that gap: the properties it must achieve, the roles it defines, its core abstractions, the protocol in full detail, and the architectural decisions that shaped the design.

3.1 Goals

Saturn pursues four design goals, each responding to a problem identified in Chapters 1 and 2.

End users should gain AI access by simply joining a network—no software installation, no account creation, no API key entry, no model selection. The configuration effort required of consumers should be zero if Saturn achieves its primary goal.

The protocol makes no assumptions about what provides AI inference. Any service that exposes an OpenAI-compatible REST API participates equally, whether it runs on a cloud provider, a local GPU, or a travel router. Saturn binds to an interface standard,

not a provider or deployment topology.

Rather than distributing configuration burden across every user and every application, one administrator configures services for all users on a network. A university IT department, a hobbyist running a local model at home, or a household member sharing cloud API access with family—each configures once, and every device on the network benefits. This shifts total configuration effort from scaling with the number of users and applications to scaling only with the number of services.

When AI services require API keys, those keys must resist theft. A participant who joins a network, captures a broadcast credential, and leaves should find that credential expired by the time they attempt to use it elsewhere. Saturn achieves this through forced rotation: cloud-facing services rotate credentials on a fixed schedule and announce both the key and its rotation interval, constraining the window an attacker has to exploit a captured credential.

3.2 Audiences

Four roles interact with Saturn. Each encounters a different surface of the protocol.

LLM inference providers operate the AI services that Saturn discovers. Providers such as OpenAI, Anthropic, and OpenRouter already expose OpenAI-compatible REST APIs; Saturn requires no changes to their endpoints. Their incentive is distribution: a provider whose API is discoverable via Saturn reaches every user on every Saturn-enabled network without negotiating individual accounts or institutional contracts.

IT administrators perform all Saturn configuration. They obtain API keys from providers, deploy beacon processes, name services, and assign priorities. Every property that makes Saturn zero-configuration for end users—automatic discovery, credential distribution, service selection—emerges from an administrator's deliberate setup.

The administrator is Saturn's primary user.

End users consume AI through applications they already use. They do not know what an API key is, what model serves their request, or that Saturn exists. Their experience mirrors AirPlay: join a WiFi network, open a compatible application, find AI services available immediately—exactly as they find printers or speakers. When Saturn works, it is invisible.

Application developers integrate Saturn discovery into their software. Their motivation is economic: they want to offer AI-powered features without subsidizing API costs for every user. By adding Saturn support, a developer delegates credential management and provider selection to the network, writing discovery code once while any Saturn-enabled network supplies the backend.

3.3 Concepts

This section defines four abstractions that the protocol specification (Section 3.4) depends on.

3.3.1 Endpoints

A Saturn endpoint is any HTTP service that implements the OpenAI-compatible REST API—the de facto standard interface for AI text generation, originally defined by OpenAI and now implemented by Ollama, OpenRouter, LiteLLM, vLLM, and dozens of other providers. The API specifies a JSON request/response format with a messages array, model identifier, and generation parameters.

Saturn adopts this existing interface rather than defining its own because most AI-consuming applications already support it. An application with a "custom OpenAI endpoint" setting works with Saturn without code changes. The protocol does not pre-

scribe who runs the endpoint or where: a cloud provider, a university data center, or a single-board computer running quantized models all qualify, provided they implement the expected HTTP paths (Section 3.4.3).

3.3.2 Beacons

A beacon is a process that announces an AI endpoint on the local network via mDNS/DNS-SD. It registers a service under `_saturn._tcp.local.`, publishes metadata in DNS TXT records, and responds to discovery queries. A beacon does not proxy API traffic—clients connect directly to the advertised endpoint. The beacon tells them where to connect and how to authenticate; it never handles their requests.

This separation matters for three reasons. First, a beacon cannot observe user prompts or model responses, because API traffic never flows through it. Second, a beacon failure does not sever existing connections; clients that have already discovered an endpoint continue communicating with it directly. Third, beacons require minimal resources—DNS registration and credential management, not request forwarding—which enables deployment on constrained hardware such as travel routers.

Saturn defines two beacon types based on deployment context. A *cloud beacon* announces a remote AI provider and distributes ephemeral API keys that grant temporary access to that provider's API. A *local beacon* announces a service running on the local network and monitors its health, deregistering the service if it becomes unreachable.

3.3.3 Priorities

Each Saturn service advertises a numeric priority in its TXT record. Lower values indicate higher preference: a service at priority 1 is selected before a service at priority 50. Clients sort discovered services by priority and connect to the lowest available value.

Priority is the administrator's policy mechanism. An IT administrator who wants users to prefer local inference sets the local beacon to priority 1 and the cloud beacon to priority 50. Users receive local inference by default and fall back to the cloud only when the local service is unavailable. Saturn uses explicit numeric priority rather than round-robin or random selection because the administrator—not the client—controls routing policy. The semantics mirror DNS SRV record priority: lower values mean higher preference, and clients process services in ascending order.

3.3.4 Ephemeral Keys

Cloud AI providers authenticate requests with API keys. When Saturn distributes access to a cloud provider, the beacon broadcasts a temporary key rather than the administrator's permanent credentials. This key has a fixed lifetime—typically 10 minutes—and the beacon generates a replacement before the current key expires, typically every 5 minutes. The overlap window ensures zero-downtime transitions: clients using the old key continue working while new clients receive the replacement. After a brief grace period, the beacon deletes the expired key from the provider's system.

This rotation schedule serves the credential transience goal. A key captured from a network broadcast expires within minutes. An attacker who records the key and leaves the network finds it revoked before they can use it elsewhere. The approach parallels Kerberos ticket distribution: time-limited credentials issued for a specific session scope, applied here at the application layer through provider key provisioning APIs.

Ephemeral keys appear only in cloud beacon deployments. Local services that require no authentication—an Ollama instance on a private network, for example—advertise no key. The TXT record schema (Section 3.4.2) distinguishes these cases through the `features` field. This mechanism requires that the AI provider support

programmatic key creation and deletion; providers without provisioning APIs cannot participate in ephemeral key distribution.

3.4 Protocol Specification

This section specifies the Saturn protocol in sufficient detail for independent implementation. Two implementors reading only this section should produce interoperable systems.

3.4.1 Service Type

Saturn services register under the DNS-SD service type `_saturn._tcp.local.`. The underscore-prefixed format follows RFC 6763 naming conventions for DNS-SD service types [9]. The `_tcp` designator indicates that clients connect via TCP after discovery. The `.local.` suffix confines discovery to the link-local multicast domain defined by mDNS [10].

3.4.2 Record Structure

Saturn uses the DNS-SD record chain described in Section 2.2.2. Three DNS record types work together for each service instance:

A **PTR record** maps the service type `_saturn._tcp.local.` to the names of all Saturn instances on the network. Clients query this record to enumerate available services.

An **SRV record** for each instance provides a hostname and port number. The hostname resolves to an IP address via mDNS.

A **TXT record** for each instance carries key-value metadata: deployment type, API

compatibility, priority, and—for cloud beacons—ephemeral credentials. Clients select among discovered services using TXT metadata alone, without additional network requests beyond the initial DNS-SD queries.

Table 3.1 specifies the TXT record fields that constitute the Saturn protocol.

Table 3.1: Saturn TXT Record Schema

Field	Presence	Description
`version`	Required	Protocol version (currently `1.0`).
`deployment`	Required	`cloud` for remote providers; `local` for on-network inference.
`api_type`	Required	API compatibility identifier (e.g., `openai`, `ollama`).
`api_base`	Required	Base URL for API requests (e.g., `http://192.168.1.5:11434/v1` for local services; `https://openrouter.ai/api/v1` for cloud beacons).
`priority`	Required	Numeric routing preference; lower values indicate higher priority.
`features`	Required	Feature flags: `ephemeral_auth` for cloud beacons with key rotation; `network_proxy` for local services.
`ephemeral_key`	Beacon	Temporary API credential; present when `features` includes `ephemeral_auth`. Must fit within 240 usable bytes of the 255-byte TXT string limit.
`rotation_interval`	Beacon	Key rotation interval in seconds; present alongside `ephemeral_key`.

Administrators may attach additional metadata beyond the fields in Table 3.1. The protocol defines optional extensions for model enumeration (`models`), capability advertisement (`capabilities`), context window size (`context`), and pricing tier (`cost`). Clients ignore unrecognized fields. All values are UTF-8 encoded strings, and each key-value pair must independently satisfy the 255-byte DNS TXT string limit described in Section 2.2.1.

3.4.3 Endpoint Expectations

Every Saturn endpoint must implement three HTTP paths, listed in Table 3.2.

Table 3.2: Required Saturn Endpoint Paths

Method	Path	Behavior
`GET`	`/v1/health`	Returns a JSON object with a `status` field. Clients issue this request to verify service availability before and during use.
`GET`	`/v1/models`	Returns a JSON array of available model identifiers in the OpenAI list-models format.
`POST`	`/v1/chat/completions`	Accepts a JSON body containing a `messages` array and `model` identifier. Returns a chat completion response. Must support both streaming (server-sent events) and non-streaming modes.

For cloud beacons, the `api_base` TXT field points to the upstream provider's API rather than to the beacon itself. Clients that discover a cloud beacon extract the `ephemeral_key` from the TXT record and authenticate directly with the provider at the advertised `api_base`.

3.4.4 Discovery Process

Client discovery follows four steps:

Step 1: Browse. The client multicasts a DNS-SD PTR query for `_saturn._tcp.local.`, requesting the names of all Saturn service instances on the local network.

Step 2: Resolve. For each discovered instance, the client queries the corresponding SRV and TXT records. The SRV record yields a hostname and port; the TXT record provides metadata including deployment type, API base URL, priority, and—for cloud

beacons—an ephemeral credential.

Step 3: Sort. The client orders discovered services by the `priority` field in ascending order. Ties may be broken by implementation-defined criteria such as the `cost` field or random selection.

Step 4: Select. The client issues a `GET /v1/health` request to the highest-priority service. If the service responds, the client connects. If not, the client proceeds to the next service in priority order. Discovery fails only when no service responds.

When a connected service becomes unavailable—detected by a failed health check or a failed API request—the client re-executes the discovery process from Step 1.

3.5 Architecture Decisions

Four architectural decisions shape Saturn. Each represents a deliberate choice against a viable alternative.

Rather than routing traffic through an intermediary, Saturn announces where AI services live and lets clients connect directly. A proxy architecture would simplify client integration—clients need only one URL—but introduces a single point of failure, grants the operator visibility into every user prompt and response, and must handle the entire throughput of the backends it fronts. The beacon pattern requires clients to discover endpoints, but eliminates these liabilities. Discovery is automatable: clients execute it once at startup and repeat only on service failure, making the trade-off acceptable.

Saturn uses link-local multicast for discovery rather than a centralized service catalog. A registry server offers faster lookups and cross-subnet visibility but reintroduces infrastructure dependency: someone must operate, maintain, and secure it. mDNS requires no infrastructure beyond the network itself. This confines Saturn to a single broadcast domain—devices must share a multicast segment—but matches the deploy-

ment contexts Saturn targets: home networks, campus WiFi, and small offices where all participants share a network segment.

The protocol itself is decoupled from implementation. Saturn specifies a service type, a TXT record schema, and endpoint expectations, but does not mandate a programming language, an mDNS library, or a deployment platform. Independent implementations interoperate by construction: any process that registers under `_saturn._tcp.local.` with conforming TXT records and serves the expected endpoints is a valid Saturn service.

Saturn targets text completion and chat endpoints exclusively. Image generation, audio synthesis, and embedding computation use different API interfaces and would require extending both the TXT schema and endpoint expectations. Text completion is chosen because it is the most widely deployed AI capability and the one the OpenAI-compatible API standardizes. Establishing zero-configuration discovery on this single modality demonstrates feasibility; extending to additional modalities requires no changes to the underlying mDNS/DNS-SD mechanism, only new service types and TXT field definitions.

Chapter 4

Implementation

4.0.1 Server Architecture

The core Saturn package provides shared libraries for beacon registration, mDNS query handling, and TXT record encoding/decoding. The server architecture centers on a beacon process that registers a service under `_saturn._tcp.local.` using the system's native mDNS responder (Bonjour on macOS, Avahi on Linux) or direct UDP multicast for platforms without daemon support. The beacon maintains TXT records with current metadata, refreshes service registration periodically to prevent timeout, and handles credential rotation for cloud-facing beacons.

4.0.2 Client Discovery

Client code follows the four-step discovery process from Section 3.4.4: browse for `_saturn._tcp.local.`, resolve SRV and TXT records, sort by priority, and probe each service with a health check. The client library exposes a simple API: call `discover()` to return available services with metadata, then `connect(service)` to establish a client authenticated with the provided ephemeral key. Discovered services cache for 60 sec-

onds; clients refresh when they fail to connect to a previously discovered service.

4.1 Server Types

4.1.1 Ollama Server

Saturn's Python reference implementation includes a built-in beacon for Ollama, a widely deployed local inference engine. The Ollama beacon discovers running instances via port scanning on port 11434 or explicit configuration, verifies they respond to health checks, and advertises them via mDNS. Models enumerated from Ollama propagate directly into Saturn TXT records, keeping model lists synchronized automatically.

4.1.2 OpenRouter Server

The OpenRouter beacon provisions cloud-based inference by distributing ephemeral API keys from the administrator's OpenRouter account. OpenRouter supports programmatic key creation and deletion, enabling Saturn's credential rotation strategy. The beacon authenticates to OpenRouter's key provisioning API, creates 10-minute-lifetime keys on a schedule, and broadcasts each key in TXT records until replacement arrives.

4.1.3 Fallback Server

A fallback beacon serves requests directly, returning hardcoded models and demonstration responses. This server serves testing and documentation purposes, enabling developers to validate Saturn discovery without requiring Ollama or cloud credentials.

4.2 Client Integration Patterns

4.2.1 Native SDK Integration

Applications that natively support OpenAI-compatible endpoints (e.g., Cursor, Continue, Open WebUI) require minimal integration work. The Saturn client library replaces manual endpoint configuration with a single call to `discover()`, which returns available services. Applications populate their provider dropdown with discovered services; users select one from the list rather than entering URLs and credentials manually. This pattern requires only adding the Saturn library as a dependency and replacing two or three lines of configuration code.

4.2.2 Application Plugin

4.2.3 Tool Protocol Bridge

4.2.4 Subprocess Bridge

Saturn's design assumption—that applications natively support mDNS discovery—does not hold for most software. Most applications were built before Saturn existed. To prove Saturn works in production applications, we built a subprocess bridge pattern: a lightweight proxy that performs discovery on behalf of an application that cannot do it natively.

The VLC media player integration demonstrates this pattern. VLC is one of the world's most widely installed desktop applications, making it an ideal test case. End users do not have API keys, configuration files, or technical knowledge. If Saturn can bring AI discovery into VLC with zero user configuration, it proves the approach scales beyond developer tools.

User Experience

Installation requires only copying a directory to VLC's extensions folder and restarting. Activation is a single menu click: View → Extensions → Saturn Chat. No environment variables, no Python installation, no API key management.

When a user activates Saturn Chat, the extension automatically extracts metadata from the currently playing media—title, artist, album, genre, current timestamp—using VLC's native metadata API. A dialog window appears with two dropdowns: one for discovered services, one for available models. The user types a question about the media and receives a styled response. All discovery and model enumeration happen automatically in the background.

This interaction pattern mirrors traditional zero-config discovery: a user plugs in a USB printer and it appears in the system's printer list without driver installation or configuration. Here, a user opens VLC and AI services appear in a dropdown without credential entry or endpoint specification.

Technical Architecture

The integration uses a two-layer architecture to work around VLC Lua's limitations. VLC's extension system supports only Lua scripting with restricted HTTP capabilities (GET requests only, no POST). Lua also lacks native mDNS support.

Layer 1: A Python/FastAPI bridge service (‘vlc_discovery_bridge.py‘) runs as a background subprocess launched by the Lua extension. The bridge performs mDNS discovery using the same ‘dns-sd‘ subprocess calls as the core Saturn client, browses ‘_saturn._tcp.local.‘, resolves service addresses, maintains health checks, and exposes an OpenAI-compatible REST API on localhost (127.0.0.1). When the Lua extension activates, it launches the bridge with automatic port assignment. The bridge writes its

port to a file that the extension reads, establishing inter-process communication without hardcoding ports.

Layer 2: The Lua extension (‘saturn_chat.lua‘) creates a VLC dialog window, manages user interaction, formats requests as URL-encoded JSON payloads (working around Lua’s GET-only limitation), and sends them to the localhost bridge via HTTP. When the user deactivates the extension, the bridge is terminated.

The bridge is packaged into a standalone executable using PyInstaller, eliminating the Python dependency for end users. Cross-platform backgrounding handles Windows (‘start /B‘), macOS, and Linux process launching. The entire installation is a directory copy; deployment requires no system-level configuration.

Significance

The VLC integration serves three purposes in Saturn’s thesis argument. First, it proves that AI provisioning can happen at the network layer like print services—not merely in CLI tools or developer IDEs, but in applications designed and used by non-technical people. Second, the VLC bridge uses identical mDNS discovery code as all other Saturn components: ‘_run_dns_sd_discovery()‘ calls ‘dns-sd -B _saturn._tcp local‘ and ‘dns-sd -L‘, exactly matching the core Python package’s simple chat client. This validates that the Saturn protocol is application-agnostic. Third, the subprocess bridge demonstrates how to add Saturn discovery to legacy software without modifying the application itself—a pattern that extends Saturn’s reach beyond applications Saturn developers can control.

4.3 GL.iNet Router Deployment

4.3.1 Hardware Constraints

Travel routers like the GL.iNet MT300N-V2 run OpenWRT with severely constrained resources: 580 MB RAM, 8 GB flash storage, and a single-core processor. Traditional AI inference is impossible on this hardware. Beacon operation, however, requires only mDNS registration and credential rotation—both lightweight operations measurable in kilobytes and milliseconds. A Saturn beacon on this hardware advertises remote AI endpoints and distributes ephemeral keys, never inferring locally. This enables the router to function as a network provisioning device: DHCP assigns IP addresses, DNS provides name resolution, and Saturn provides AI service discovery.

4.3.2 OpenWRT Integration

Saturn integrates into OpenWRT as a custom package. The beacon runs as a system service, starting automatically on boot. Administrator configuration occurs through UCI (Unified Configuration Interface), OpenWRT's native configuration system. An administrator specifies cloud provider credentials and local service URLs; the beacon validates them at startup and advertises the resulting services. The package includes minimal dependencies—pure Python with only standard library imports—fitting within the constrained storage budget.

4.3.3 Significance

The GL.iNet router demonstrates that Saturn can run anywhere network infrastructure runs. A user who administers their home WiFi network can now provision AI services

alongside IP assignment, DNS, and DHCP. This flattens the distinction between infrastructure and application: the same device that has always configured the network now configures AI access. For users sharing household WiFi or small offices sharing LANs, this is transformative. Every device that joins the network automatically discovers AI services.

4.3.4 Limitations

The router cannot serve as a hub for multi-site deployments. A user with two homes (weekday and weekend) still needs separate configurations. Router failure means lost service discovery until the device restarts. Deployment at scale (multi-building campus) requires multiple routers or centralized infrastructure. These limitations reflect the design constraints of edge deployment, not failures specific to Saturn.

Chapter 5

Evaluation

[Chapter pending rewrite.]

Chapter 6

Discussion

[Chapter pending rewrite.]

Chapter 7

Conclusion

[Chapter pending rewrite.]

Appendix A

Saturn Protocol Specification

This appendix specifies the Saturn protocol as a standalone reference. An implementor reading only this appendix and the RFCs it cites should produce a system interoperable with any conforming Saturn deployment.

A.1 Service Registration

A Saturn beacon registers one DNS-SD service instance under the type `_saturn._tcp.local.` (RFC 6763 [9]). Registration creates three DNS records:

1. A **PTR record** mapping `_saturn._tcp.local.` to the instance name (e.g., `ollama-lab._saturn._tcp.local.`).
2. An **SRV record** providing the beacon's hostname and port.
3. A **TXT record** carrying service metadata as key-value pairs.

The instance name is a human-readable UTF-8 string chosen by the administrator. It must be unique within the multicast domain. The `.local.` suffix confines all queries and responses to the link-local scope defined by mDNS (RFC 6762 [10]).

A.2 TXT Record Schema

Table A.1 defines all TXT record fields. Each key-value pair is encoded as a single DNS TXT string (`key=value`) and must not exceed 255 bytes (RFC 6763, Section 6.1). Clients must ignore unrecognized keys.

A.3 Endpoint Requirements

Every URL advertised in `api_base` must implement three HTTP paths:

`GET /v1/health` Returns JSON with a `status` field. Clients call this to verify liveness before use and periodically during sessions. A non-200 response or timeout (recommended: 5 seconds) indicates the service is unavailable.

`GET /v1/models` Returns a JSON object with a `data` array of model objects, each containing at minimum an `id` field. Follows the OpenAI list-models response format.

`POST /v1/chat/completions` Accepts a JSON body with `model` (string) and `messages` (array of {`role, content`} objects). Returns a chat completion response. Must support both non-streaming (JSON response) and streaming (server-sent events with `data:` prefix) modes, selected by the `stream` boolean in the request body.

For cloud beacons, `api_base` points to the upstream provider, not the beacon. Clients authenticate with the provider using the `ephemeral_key` from the TXT record.

A.4 Discovery Algorithm

Listing A.4 specifies client discovery in pseudocode.

```
function discover():
    instances = mdns_browse("_saturn._tcp.local.")
    services = []
    for instance in instances:
        srv = mdns_resolve_srv(instance)
        txt = mdns_resolve_txt(instance)
        if txt["version"].major != 1:
            skip  // incompatible protocol version
        services.append({
            host: srv.hostname,
            port: srv.port,
            metadata: txt
        })
    sort services by metadata["priority"] ascending
    for service in services:
        url = service.metadata["api_base"]
        headers = {}
        if "ephemeral_auth" in service.metadata["features"]:
            headers["Authorization"] =
                "Bearer " + service.metadata["ephemeral_key"]
        response = http_get(url + "/v1/health",
                            headers, timeout=5s)
        if response.status == 200:
            return service  // connected
    return null  // no healthy service found
```

Clients may cache discovery results. A recommended cache TTL is 60 seconds; clients should re-browse when the cached service fails a health check.

A.5 Ephemeral Key Rotation Protocol

Cloud beacons rotate API credentials on a fixed schedule to bound exposure windows. The rotation proceeds as follows:

1. At startup, the beacon creates a new API key via the provider's provisioning API and records its creation time t_0.
2. The beacon publishes the key in the `ephemeral_key` TXT field and sets `rotation_interval` to the rotation period R (default: 300 seconds).
3. At $t_0 + R$, the beacon creates a new key k_1, updates the TXT record to advertise k_1, and starts a grace timer for the old key k_0.
4. After a grace period G (default: 5 seconds), the beacon deletes k_0 from the provider's system.
5. Steps 3–4 repeat every R seconds.
6. On shutdown, the beacon deletes the current key from the provider.

The key lifetime L (default: 600 seconds) exceeds the rotation interval R, so at most two keys are valid simultaneously during the overlap window. This guarantees zero-downtime transitions: clients using the previous key continue working while the TXT record advertises its replacement.

A.6 Conformance

A conforming Saturn beacon must:

- Register exactly one `_saturn._tcp.local.` service instance per advertised endpoint.
- Include all six required TXT fields (Table A.1).
- Include `ephemeral_key` and `rotation_interval` when `features` contains `ephemeral_auth`.
- Deregister the mDNS service on shutdown.
- Delete active ephemeral keys from the provider on shutdown.

A conforming Saturn client must:

- Discover services via mDNS PTR query for `_saturn._tcp.local.`.
- Parse all required TXT fields; ignore unrecognized fields.
- Sort discovered services by `priority` ascending.
- Verify service health via `GET /v1/health` before use.
- Fall back to the next-priority service when the current service becomes unreachable.

Table A.1: Complete TXT Record Field Reference

Field	Presence	Type	Semantics
`version`	Required	String	Protocol version. Currently `1.0`. Clients must reject services with unrecognized major versions.
`deployment`	Required	Enum	`cloud` (remote API provider) or `local` (on-network inference server).
`api_type`	Required	String	API compatibility identifier: `openai`, `ollama`, or provider-specific values.
`api_base`	Required	URL	Base URL for API requests. Local example: `http://192.168.1.5:11434/v1`. Cloud example: `https://openrouter.ai/api/v1`.
`priority`	Required	Integer	Routing preference. Lower values indicate higher priority. Clients sort ascending; ties broken by implementation.
`features`	Required	String	Comma-separated feature flags. Defined values: `ephemeral_auth` (key rotation active), `network_proxy` (local proxy service).
`ephemeral_key`	Conditional	String	Temporary API credential. Required when `features` includes `ephemeral_auth`. Must fit within 240 usable bytes.
`rotation_interval`	Conditional	Integer	Key rotation period in seconds. Required alongside `ephemeral_key`. Default: 300 (5 minutes).
`models`	Optional	String	Comma-separated model identifiers available at this endpoint.
`capabilities`	Optional	String	Supported modalities: `chat`, `completion`, `embedding`.
`context`	Optional	Integer	Maximum context window in tokens.
`cost`	Optional	String	Pricing tier: `free`, `metered`, or `unknown`.

Appendix B

Generative AI in Thesis Development

B.1 Knowledge Graph Authorship

Saturn research spans twenty papers from five domains (networking, security, HCI, AI accessibility, threat modeling), six implementation components touching multiple thesis claims, and three core claims with overlapping evidence. A traditional document cannot efficiently track these relationships: which papers support claim-1 versus claim-3? Where do we contrast with existing work? How do concepts like mDNS and ephemeral keys nest logically? Which components demonstrate which claims?

We built the moons knowledge graph to solve this problem—a queryable structure of lightweight nodes (concept, claim, paper, component, chapter) with typed edges (supports, contrasts, related, demonstrated$_{by}$).$Thegraphitselfcomprisesapproximatelyfivehundredlinesc$
$whatdistinguishes$“$supports''(directevidence)from$“$related''(connectionexistsbutweaker)?Thisprecisior$

In Phase 2, Claude generated candidate edges for each claim—roughly one hundred suggestions in minutes, work that would have required hours of manual research. In Phase 3, we iteratively refined the graph: verifying relationships against source papers, rewriting auto-generated descriptions for accuracy, flagging contradictions. In Phase

4, moons became a living knowledge base, updated whenever new papers arrived or claims shifted.

Claude accelerated scaffolding through fast edge generation, cross-domain synthesis (connecting mDNS papers to AI accessibility literature), consistency checking, and iteration. Claude did not verify claim accuracy. Determining whether Siddiqui 2012 truly supports claim-1 required us to read the paper ourselves. Resolving contradictions between papers required human judgment. Defining what "supports" means for Saturn was our decision; Claude executed it. Checking descriptions against source material remained our responsibility. The result: moons maps Saturn's evidence, authored by humans with AI acceleration. Every edge is human-verified; every description reflects domain understanding we applied after Claude's draft.

B.2 Iterative Advisory Role

Starting in January 2026, we established an ongoing conversation with Claude in which we described ideas, drafts, or problems; Claude asked clarifying questions and offered suggestions; we evaluated suggestions in Saturn's context and iterated. This is not Claude writing the thesis. It is Claude helping us think through it.

Claude served three advisory roles. First, conceptual feedback. When we described claim-2 as "reducing configuration effort," Claude asked "compared to what?" We realized we hadn't specified a baseline. Clarifying it as the status quo (developers using cloud APIs directly, managing keys manually, no local fallback) pushed us to examine whether our evaluation measured against this comparison.

Second, argument coherence. Claim-1 assumes routers announce via mDNS; claim-3 includes a threat model where an attacker lies about a service. Claude asked: if announcements are just broadcasts, how can we prevent the lie? We realized our threat

model didn't clearly separate infrastructure-level guarantees (we provide none) from client-side validation and ephemeral keys. Clarifying this boundary strengthened our security argument.

Third, writing support. When we described the intro paragraph's target audience (practitioners building AI applications), Claude suggested framing the problem not as "manual configuration is error-prone" but as "developers face a choice: manual setup (time-consuming, error-prone) or cloud APIs (expensive, requires internet)." We adopted and adapted this framing.

Claude also made recommendations we declined. When we described claim-2, Claude suggested adding a fourth claim about cost savings. We rejected this because Saturn is framed as equity—access, not efficiency. A fourth claim would dilute focus. When we planned evaluation methodology, Claude suggested Monte Carlo simulation to predict large-scale performance. We chose real trials with a physical router instead; simulation is faster but riskier if parameter estimates are wrong. When we reviewed the threat model, Claude suggested including quantum computing threats to make the work future-proof. We focused on classic attack vectors; quantum threats are speculative for our scope.

Claude also proposed technical features that required correction. Claude suggested adding DRM to prevent unauthorized model access; we rejected this because Saturn targets resource-constrained environments where DRM increases deployment friction. In other cases, Claude's suggestions revealed gaps in our own documentation—prompting us to clarify design decisions we had not yet written down.

The pattern holds: Claude generates options and suggestions; we evaluate them against Saturn's design, goals, and constraints. We made every final decision.

B.3 Limits and Human Judgment

Several recommendations were declined—the fourth claim, Monte Carlo simulation, and quantum threats—each illustrating the boundary between AI suggestion and human judgment.

The pattern: Claude excels at generating options. We evaluate them in context—Saturn's goals, constraints, timeline.

This thesis would exist without Claude. It would be slower: organizing twenty papers without structured support takes longer. It would be less coherent: iterative feedback caught inconsistencies we wouldn't have noticed alone. It would be more isolated: cross-domain synthesis didn't happen naturally. It would be less defended: critique from Claude, even from an AI, sharpened our arguments. But the intellectual contribution—design choices, claim framing, evidence evaluation, judgment calls—is ours.

Claude was a tool for synthesis and refinement, not a proxy for thinking. We document this openly because Saturn argues that generative AI should be accessible to everyone. Concealing AI's role in our own work would contradict that argument.

Bibliography

[1] Anthropic. Claude pricing, 2025.

[2] Anysphere. Cursor: The AI Code Editor, 2025.

[3] Apple Inc. Bonjour, 2002.

[4] Elisa Bassignana, Amanda Cercas Curry, and Dirk Hovy. The ai gap: How socioeconomic status affects language technology interactions. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18647–18664, 2025.

[5] R. Braden. Requirements for internet hosts – communication layers. RFC 1122, 1989.

[6] Valerio Capraro, Austin Lentsch, Daron Acemoglu, et al. The impact of generative artificial intelligence on socioeconomic inequalities and policy making, 2024.

[7] Víctor D. Carmona-Galindo, Hou Ung, Manhao Zeng, Christine Broussard, Elizaveta Taranenko, Yousef Daneshbod, David Chappell, and Todd Lorenz. Generative ai as a sociotechnical challenge: Inclusive teaching strategies at a hispanic-serving institution. *Knowledge*, 5(3):18, 2025.

[8] S. Cheshire, B. Aboba, and E. Guttman. Dynamic Configuration of IPv4 Link-Local Addresses. RFC 3927, May 2005.

[9] S. Cheshire and M. Krochmal. DNS-Based Service Discovery. RFC 6763, February 2013.

[10] S. Cheshire and M. Krochmal. Multicast DNS. RFC 6762, February 2013.

[11] Continue. Continue: Open-Source AI Code Assistant, 2025.

[12] Carlos J. Costa, Manuela Aparicio, Sofia Aparicio, and Joao Tiago Aparicio. The

democratization of artificial intelligence: Theoretical framework. *Applied Sciences*, 14(18):8236, 2024.

[13] Digital Living Network Alliance. DLNA Guidelines, 2017. DLNA merged with SpireSpark International in 2017.

[14] R. Droms. Dynamic Host Configuration Protocol. RFC 2131, March 1997.

[15] Sonja Gabriel. Generative ai and educational (in)equity. In *Proceedings of the 4th International Conference on AI Research (ICAIR 2024)*, pages 133–142, 2024.

[16] NetBIOS Working Group, Defense Advanced Research Projects Agency, Internet Activities Board, and End to End Services Task Force. Protocol standard for a netbios service on a tcp/udp transport: Concepts and methods. RFC 1001, March 1987.

[17] Erik Guttman. Autoconfiguration for ip networking: Enabling local communication. *IEEE Internet Computing*, 5(3):81–86, 2001.

[18] Jan.ai. Jan: Open-source chatgpt alternative, 2025.

[19] Jordi Bruin. MacWhisper: Transcription Using OpenAI's Whisper, 2025.

[20] Daniel Kaiser and Marcel Waldvogel. Adding privacy to multicast dns service discovery. In *IEEE 13th International Conference on Trust, Security and Privacy in Computing and Communications*, pages 809–816, 2014.

[21] Daniel Kaiser and Marcel Waldvogel. Efficient privacy preserving multicast dns service discovery. In *IEEE International Conference on High Performance Computing and Communications (HPCC)*, pages 1261–1268, 2014.

[22] Bastian Könings, Christoph Bachmaier, Florian Schaub, and Michael Weber. Device names in the wild: Investigating privacy risks of zero configuration networking. In *IEEE 14th International Conference on Mobile Data Management*, pages 51–56, 2013.

[23] LibreChat. LibreChat: Enhanced ChatGPT Clone, 2025.

[24] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How bad can it git? characterizing secret leakage in public github repositories. In *Proceedings of the 2019 Network and Distributed System Security Symposium (NDSS)*, 2019.

[25] Microsoft. Visual Studio Code Remote Development, 2025.

[26] OASIS. Web Services Dynamic Discovery (WS-Discovery) Version 1.1. Technical report, OASIS, July 2009.

[27] Ollama. Ollama: Run Large Language Models Locally, 2025.

[28] Ollama Community. mdns service discovery pull request #751, 2024.

[29] Open Connectivity Foundation. UPnP Device Architecture 2.0. Technical report, Open Connectivity Foundation, 2015.

[30] Open WebUI. Open webui, 2025.

[31] OpenAI. Chatgpt plus, 2025.

[32] Betsy Beyer Osborn, Jeff Peck, Luca Ward Petazzoni, et al. Beyondcorp: Design to deployment at google. volume 41, pages 28–34, 2016.

[33] Lennart Poettering et al. Avahi: A Free Zero-Configuration Networking (Zeroconf) Implementation, 2004.

[34] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust Speech Recognition via Large-Scale Weak Supervision, 2022.

[35] Farhan Siddiqui, Sherali Zeadally, Thabet Kacem, and Scott Fowler. Zero configuration networking: Implementation, performance, and security. *Computers and Electrical Engineering*, 38(5):1129–1145, 2012.

[36] Aleksandar Siljanovski, Anuj Sehgal, and Jürgen Schönwälder. Service discovery in resource constrained networks using multicast dns. In *Proceedings of the IEEE International Conference*, 2014.

[37] Lewis L. Strauss. Speech to the national association of science writers, 1954. New York City, September 16, 1954.

[38] SuperWhisper. SuperWhisper: AI Voice to Text for macOS, 2025.

[39] Naeem Syed, Adnan Anwar, Zubair Baig, and Sherali Zeadally. Artificial intelligence as a service (aiaas) for cloud, fog and the edge: State-of-the-art practices. *ACM Computing Surveys*, 57(8):211:1–211:36, 2025.

[40] Rory Ward and Betsy Beyer. Beyondcorp: A new approach to enterprise security. *;login:*, 39(6):6–11, 2014.

[41] Wispr. Wispr Flow: AI Dictation That Writes How You Speak, 2025.